

002: Shared-Backbone Step-wise PPO 技术路线

目标

本文给出 Step-wise PPO 的确定实现路线。critic 不作为独立模型维护，而是在 actor language model 上额外挂一个 scalar value head。actor 负责生成 action，value head 在每个环境 step 的 action 边界预测一个 state value。

Value Head 设计

设 $h_t \in \mathbb{R}^d$ 是 action 生成前边界位置的 hidden state。主版本使用两层 MLP：

$$v_\psi(h_t) = W_2 \text{GELU}(W_1 h_t + b_1) + b_2.$$

linear head 作为低成本 ablation 保留。value head 在 FSDP wrapping 前挂载，使其参与 actor optimizer 和 checkpoint；同步到 vLLM/SGLang rollout engine 前，需要过滤包含 `value_head` 的参数。

默认 value position 是

$$b_t = n_t - L_t - 1,$$

其中 n_t 是 context+response 总长度， L_t 是 response 长度。value 定义为

$$V_{\theta,\psi}(s_t) = v_\psi(H_t[b_t]).$$

这个位置不 attend 到生成 token，因此是严格 state-value，而不是 action-conditioned value。

Rollout 必需字段

每个 step sample 需要保存以下字段：

- `uid`: 初始任务 group id，用于 episode-level GRPO normalization。
- `traj_uid`: trajectory id，用于恢复 episode 内 step 序列。
- `step_idx`: 环境 step index，用于按时间顺序计算 GAE。
- `step_sample_uid`: 可选字段，用于 batch expansion/filter 后追踪样本。

Tensor 侧需要保存 response mask、token-level reward、immediate step reward、old log probability、old scalar state value、token-broadcast advantage 和 scalar step return。

Forward 路径

actor forward 支持 `return_step_values=True`。启用后, 模型返回语言模型输出和 token-wise scalar values; StepPPO actor 再根据 `value_position` 取一个 scalar。old-log-probability 阶段记录:

$$(\log \pi_{\theta_{\text{old}}}, V_t^{\text{old}}).$$

actor update 阶段重新计算:

$$(\log \pi_{\theta}, V_t).$$

Step-wise Advantage

对每个 `traj_uid`, 按 `step_idx` 排序后计算:

$$\begin{aligned} \delta_t &= r_t + \gamma(1 - d_t)V_{t+1}^{\text{old}} - V_t^{\text{old}}, \\ A_t^{\text{step}} &= \delta_t + \gamma\lambda(1 - d_t)A_{t+1}^{\text{step}}, \quad R_t^{\text{step}} = A_t^{\text{step}} + V_t^{\text{old}}. \end{aligned}$$

这些量全部是 environment-step indexed, 不是 token indexed。

Episode Relative Signal

对 task group g 内 trajectory i , 定义 episode return R_i^{epi} 。GRPO-style signal 为

$$A_i^{\text{epi}} = \frac{R_i^{\text{epi}} - \mu_g}{\sigma_g + \epsilon}.$$

该 scalar 会广播到 trajectory i 的所有 step。

Final Advantage 版本

当前 v1 版本为

$$A_t^{\text{final}, \text{v1}} = \frac{\hat{A}_t^{\text{epi}} + w\hat{A}_t^{\text{step}}}{1 + w}.$$

v0 base 版本为

$$A_t^{\text{final}, \text{v0}} = \frac{\hat{A}_t^{\text{epi}} + wA_t^{\text{step}}}{1 + w}.$$

二者区别在于 step GAE advantage 是否先做 batch-level whitening。

Policy 与 Value Loss

actor 使用 PPO clipped objective。对 step t 中 token ℓ :

$$\rho_{t,\ell}(\theta) = \exp(\log \pi_{\theta}(y_{t,\ell}) - \log \pi_{\theta_{\text{old}}}(y_{t,\ell})).$$

policy loss 为

$$\mathcal{L}_{\text{pg}} = -\mathbb{E} \left[\min(\rho_{t,\ell} A_t^{\text{final}}, \text{clip}(\rho_{t,\ell}, 1 - \epsilon, 1 + \epsilon) A_t^{\text{final}}) \right].$$

value loss 使用 clipped scalar regression:

$$\begin{aligned} \tilde{V}_t &= V_t^{\text{old}} + \text{clip}(V_t - V_t^{\text{old}}, -\epsilon_v, \epsilon_v), \\ \mathcal{L}_v &= \frac{1}{2} \mathbb{E}_t \left[\max \left((V_t - R_t^{\text{step}})^2, (\tilde{V}_t - R_t^{\text{step}})^2 \right) \right]. \end{aligned}$$

总 loss 为

$$\mathcal{L} = \mathcal{L}_{\text{pg}} + c_v \mathcal{L}_v + \beta \mathcal{L}_{\text{KL}} - c_H \mathcal{H}.$$

验证计划

- 检查每个 step sample 都有 uid、traj_uid、step_idx。
- 检查 batch shuffle 后 GAE 的 temporal ordering 是否正确。
- 先跑 2GPU smoke，确认日志里出现 state_values、step_returns、actor/value_loss。
- 再跑 4GPU WebShop 单 seed，与 GiGPO 和 GRPO 对比。

主要风险

最大风险是 value target 噪声通过 shared backbone 干扰 actor 表征。第二个风险是额外 value forward 带来的运行开销。需要用 detached-backbone、lower-value-coef 和 microbatch ablation 来定位。